

Alzikrayat — Architecture Report

رحيق عبد العظيم محمد الحسن

قسم تقانة المعلومات

رابط: <https://github.com/raheeg007-creator/project>

1. Project Overview:

Alzikrayat is a photo-sharing web application that allows registered users to upload photos, browse a shared gallery, and comment on each other's uploads. The project was built entirely in raw PHP, without any backend framework (Laravel, Symfony) or ORM (Eloquent, Hibernate), as required by the course specification. The goal was to implement the underlying mechanics of a web application — routing, session handling, database access, and authentication — by hand, rather than relying on a framework to provide them.

2. Architecture & Project Structure:

Architecture:

The system follows the Model-View-Controller (MVC) pattern layered on top of a physical 3-tier structure. The two structures map onto each other as follows:

3-Tier Layer	MVC Component	Implementation
Presentation	Views	PHP templates in views/, styled with Bootstrap
Application / Business	Controllers	AuthController, PhotoController, CommentController
Data	Models	User, Photo, Comment (raw PDO queries)

All controllers extend an abstract base Controller class that provides shared helpers (view rendering, redirects, HTTP method checks). All models extend a base Model class that supplies a shared PDO connection from a Database singleton. At the core of the system is a hand-written Router: routes are registered with an HTTP method, a path pattern, and a [Controller, action] pair. At request time, {param} placeholders in each route are converted into a regular expression capturing group, for example:

```
'/photo/{id}' -> '#^/photo/([^/]+)$#'
```

Captured values are passed directly as arguments to the matched controller action. If no route matches, the Router serves a styled 404 page.

Project structure:

```
'''
project/

├─ config/           # Database connection
├─ core/             # Base Router, Model, Controller
├─ controllers/      # Auth, Photo, Comment
├─ models/           # User, Photo, Comment
├─ views/            # Pages
├─ routes/           # Route registration
└─ public/           # Entry point + uploaded images
'''
```

3. Database Design:

The schema consists of three normalized tables: users, photos, and comments. Both photos and comments hold a foreign key back to users, and comments additionally reference photos — both with ON DELETE CASCADE, so deleting a user or a photo automatically removes its dependent records. All primary keys are auto-incrementing integers, and every query is written by hand using PDO prepared

statements with named placeholders (e.g. :email), which is the primary defense against SQL injection.

4. Main Features:

- User registration and login with Bcrypt-hashed passwords (password_hash()); raw passwords are never stored.
- Session-based authentication with a dynamic navbar ('Hi <name>' / 'Please Login').
- A separate 'last login' cookie, independent of the session, valid for 7 days.
- Photo upload with title and description, validated and stored in public/images/uploads/.
- Gallery with switchable display styles (3-column, 4-column, list).
- Photo deletion restricted to the owner, enforced at the SQL query level (WHERE id = :id AND user_id = :userId).
- Commenting system, updating the photo detail page immediately after submission.
- Three validation layers on all forms: HTML5, JavaScript, and server-side (the only authoritative layer).
- XSS protection via htmlspecialchars() on all dynamic output.

5. Novelty / Advanced Features:

5.1 Live Gallery Search by Title:

A search box was added above the gallery (#gallerySearch). As the user types, a JavaScript keyup listener filters the photos that are already rendered on the page: it loops over every .gallery-item element, compares the typed text against each photo's title (case-insensitive), and toggles the visibility of non-matching items. No request is sent back to the server — the filtering happens entirely client-side, against the elements already present in the DOM, which makes the results feel instant.

5.2 Visual Photo Filters:

On the photo detail page, a row of filter buttons (Normal, B&W, Sepia, Vivid, and a custom warm-tone filter) lets the viewer change how the photo appears without modifying the stored file. Each button calls an applyFilter(value, button)

JavaScript function that sets the CSS 'filter' property on the displayed `<img>` element directly — for example `grayscale(100%)` for black-and-white, or a combination such as `sepia(40%) hue-rotate(315deg) contrast(110%)` for a custom warm tone. The function also toggles an 'active' class so the currently selected filter button stays highlighted. Because the filter is applied purely through CSS, the original uploaded image on disk is never altered; the effect is local to that viewer's browser session.

6. Conclusion:

Building Alzikrayat without a framework required implementing routing, authentication, and data access manually, which gave a much clearer understanding of how these mechanisms work beneath the abstractions that frameworks normally provide. The strictest constraints — the regex-based router and the ownership check embedded directly in SQL queries — turned out to be the most instructive parts of the project, and the ones most worth being able to explain in detail.